Model Size and a Data-Driven Dropout Follow-up in Piano Music-Event Transformers

Teo Chloe Student number: A0286502E DSA4212
Repository: https://github.com/chloeheh/DSA4212-project-github

Summary. We compare three randomly initialized GPT models under two hours of A10 training time per run. Small performs best. Rising validation losses in medium and large motivate a follow-up with stronger dropout. This improves those models but does not reverse the ranking. We report the unsuccessful change for small, the assumptions behind final testing, and the limits of these single-seed results.

1 Data and prediction task

We use the MIDI-only MAESTRO v3.0.0 piano performances [1]. The supplied training/validation/test splits contain 962/137/177 pieces and 33,293,600/3,790,187/4,334,269 event tokens. The documentation keeps performances of the same composition in one split [2]; we preserve this split but do not independently audit every possible overlap.

The tokenizer converts music into 376 instructions: 128 note starts, 128 note endings, 100 time shifts, 16 velocity bins, two pedal states, and BOS/EOS boundaries. Timing is rounded to 10 ms; longer pauses use multiple shifts of at most one second. At equal timestamps, note endings precede pedal changes and note starts. A velocity token precedes every note start. Only pedal state changes are emitted. This preserves expressive event order while discarding some timing/velocity detail and other MIDI metadata. A token is not a note, and 1,024 tokens cover different musical durations in busy and quiet passages.

The notebook checks token ranges, lengths, and piece boundaries and decodes one source piece to inspect the tokenizer. This is not a generated sample. Storage details and preparation steps are in the reproduction guide; our results concern event prediction, not audio reconstruction.

2 Research question, models, and evaluation criteria

Objective. Starting with random weights, train a model to predict the next music event and find the *best performance within a fixed training-time budget per run.* This is a controlled study of model size, rather than an attempt to build a production music generator. The Smol Training Playbook recommends defining the purpose before choosing a model [3]; we use this practitioner guide for framing, not as peer-reviewed evidence for performance.

Initial hypothesis. A larger model will achieve lower validation loss because it can learn more complex musical patterns. We test this under the same two-hour GPU-time budget for each size. This is a hypothesis to check, rather than an expected result we must obtain. The initial comparison concerns model size; the follow-up question is developed from the baseline observations in Sections 6–7.

Table 1: Dense decoder-only GPT variants. Counts include learned position embeddings and count tied token/output weights once.

Model	Layers	Heads	Width	Total parameters
Small	6	4	256	5,080,320
Medium	8	8	512	25,891,328
Large	12	12	768	86,029,056

Each run uses one Modal NVIDIA A10 GPU with 24 GB memory and BF16 support [4]; models run sequentially. All use vocabulary 376, context 1,024, and effective batch 32. Small/medium/large are project-specific labels, not pretrained GPT-2 variants.

Primary metric and checkpoint selection

For target event y_i and its earlier context c_i, the average negative log likelihood (cross-entropy loss) and perplexity are

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(y_i \mid c_i), \qquad \text{PPL} = \exp\{\mathcal{L}(\theta)\}. \tag{1}$$

Loss uses natural logarithms and is measured in nats/token. Lower loss and perplexity are better. Unlike accuracy, loss rewards assigning more probability to the correct event, even when it is not the top prediction. It gives no special credit for a musically similar wrong pitch and does not measure listening quality.

Each validation check averages 20 batches of four 1,024-token windows: 81,920 target positions, which may overlap. Resetting evaluation seeds gives all models the same windows. Dropout and gradient calculations are disabled.

A checkpoint is a saved model state. We select best.pt, the lowest recorded validation loss within the budget; ckpt.pt is the latest state. Selection uses no test loss. All runs have the full budget to improve, even if their best result occurs earlier. Final losses diagnose later overfitting. Section 9 describes full-coverage evaluation.

3 Adapting nanoGPT to MAESTRO

nanoGPT provides compact PyTorch model and training code [5]. It lets us inspect data loading, optimization, and checkpointing and vary model size without changing frameworks. We choose it for clarity and reuse, without claiming it is the fastest implementation.

The decoder learns token and absolute position embeddings. Causal attention uses earlier tokens while masking future targets [6]. Blocks use pre-layer normalization, residual connections, and GELU feed-forward networks with four times the hidden width. Token embeddings and the output head share weights. All models start from random weights.

The loader memory-maps token files and samples uniformly over valid within-piece window starts. A piece with $L > 1{,}024$ tokens provides $L - 1{,}024$ starts, so longer pieces receive more sampling opportunities. Inputs are $t_0, \ldots, t_{1023}$ and targets are $t_1, \ldots, t_{1024}$. Neither crosses piece boundaries; shorter pieces cannot supply full training windows.

The loader validates offsets, lengths, and token IDs and overrides the inherited vocabulary default with 376. Architecture configurations and environment details are recorded in the repository.

Table 2: Shared training settings; the baseline uses dropout 0.1.

Optimizer	AdamW	Weight decay	0.1
(β_1, β_2)	$(0.9, 0.95)$	Gradient clipping	Norm 1.0
Peak / minimum LR	3×10^{-4} / 3×10^{-5}	Seed	1337
Precision	BF16 autocast	Effective batch	32 sequences
Context	1,024 tokens	Targets/update	32,768

AdamW adapts updates using past gradients and separately applies weight decay [7]. Matrix weights receive decay; one-dimensional normalization parameters do not. CUDA uses fused AdamW and, when available, PyTorch scaled dot-product attention. Compilation is disabled and compiler modes were not benchmarked.

The 7,200-second loop budget includes validation and checkpoint writes. Setup and final evaluation/saving add overhead, and an ongoing update finishes at the boundary. This is approximately two hours of loop time, not exactly billed time or fixed FLOPs. Atomic checkpoints save weights, optimizer/scaler, random states, token counts, and elapsed budget. Resuming keeps the original budget; Modal Volumes persist outputs after client disconnection. The image pins Python 3.11, PyTorch 2.6.0, and NumPy 2.2.3.

4 Training faster within the time budget

Testing batch sizes while keeping 32 sequences per update

A microbatch is the set of sequences processed in one forward/backward pass. We benchmark sizes 4, 8, 16, and 32 on the A10, accumulating gradients as needed to keep 32 sequences per optimizer update. Throughput measures target tokens processed per second.

We time complete updates, including loading, transfer, forward/backward passes, clipping, and AdamW. After three warm-up steps, six measured steps give the median update time.

We choose the highest measured throughput below 85% GPU memory use. Microbatch 32 wins for all three sizes and is used in training, with BF16 and fused optimization.

This compares only the tested batch sizes. It does not establish a global optimum and excludes periodic validation/checkpoint overhead.

Table 3: A10 batching measurements. Throughput is target tokens/s; memory is peak allocated GB, not total reserved/device usage.

Model	Batch 4	Batch 32	Speedup	Batch-32 GB
Small	424,395	603,065	1.42×	2.24
Medium	151,006	182,301	1.21×	5.85
Large	57,946	68,590	1.18×	13.22

What the same time budget means

All models use the same learning-rate schedule based on elapsed time. Let $u = t/7{,}200$ be the fraction of the budget used. For the first $w = 0.02$ of the budget (about 144 seconds), the learning rate rises towards $\eta_{\max}$. After this warm-up, it follows

$$\eta(u) = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2}\left[1 + \cos\left(\pi \frac{u - w}{1 - w}\right)\right], \quad u \geq w. \tag{2}$$

The code keeps the learning rate slightly above zero during warm-up. Models therefore have similar learning rates at the same elapsed time, even though they have completed different numbers of updates. Time spent on validation and checkpoint saving also counts towards the schedule.

Table 4: Actual baseline training exposure within approximately 7,200 seconds. Equivalent passes are tokens seen divided by 33,293,600; sampling is with replacement, not ordered epochs.

Model	Updates	Targets seen (billions)	Equivalent passes
Small	127,396	4.175	125.4
Medium	40,719	1.334	40.1
Large	15,430	0.506	15.2

Small processes about 8.3 times as many targets as large. Larger capacity therefore comes with fewer updates and less data exposure. Equal A10 time answers a rental-budget question, but does not isolate model size, equalize FLOPs, or measure equal-token efficiency. Shared learning rate and dropout improve control, without establishing the best settings for every size.

5 Baseline learning curves: dropout 0.1

Appendix Figure 1 shows 13 recorded evaluations for each model: before training, roughly every ten minutes, and at the end. Training loss is measured on fixed training windows with dropout turned off. It is not the loss from a single training microbatch. The lower plots leave out the initial evaluation and use the same scale to make later changes easier to compare. This baseline comparison was used to decide what to investigate next.

Table 5: Baseline loss and best recorded time. All values are sampled cross-entropy in nats/token.

Model	Best val.	Best time (min)	Final train	Final val.
Small	**1.7652**	110.1	1.5603	1.7659
Medium	1.8018	40.1	1.3299	1.8892
Large	1.8122	50.3	1.2865	1.9224

All models improve quickly during the first ten minutes. Small then improves more slowly and levels off near the end. Medium and large reach their best validation losses earlier. After that, their training losses keep falling while their validation losses rise. Large has the lowest final training loss but the highest final validation loss. This shows that fitting the training pieces well does not necessarily mean predicting unseen pieces well.

Using the best recorded validation checkpoint, small ranks first, followed by medium and large. The differences are fairly small: medium and large have best-checkpoint perplexities about 3.7% and 4.8% higher than small. This does not mean that more parameters always hurt. It means that larger models did not perform better with this dataset, implementation, schedule, and time limit.

6 What the baseline results led us to investigate

The initial hypothesis is not supported in these runs: small has the lowest best validation loss. More surprisingly, medium and large continue improving on training data while becoming worse on validation data. Their final validation-minus-training gaps are 0.5593 and 0.6360, compared with 0.2055 for small. Goodfellow et al. explain overfitting as fitting training data well while failing to generalize to unseen data [8, Chapter 5]. This makes overfitting a likely explanation for our curves. The repeated rise in validation loss, together with falling, finite training losses, supports this interpretation more than a numerical failure. It does not prove a unique cause.

There is also a training-time trade-off: medium and large have about 5.1 and 16.9 times as many parameters as small, but receive fewer updates. Hoffmann et al. study the joint allocation of model size and training tokens under fixed FLOPs [9]. Their fresh-data text setting differs from our fixed corpus and GPU-time limit. We therefore do not apply their token-to-parameter rule or label our models "Chinchilla-optimal." Equal-token runs and learning-rate tuning would help separate capacity, exposure, and optimization effects.

Follow-up question. After seeing these curves, we asked whether stronger regularization could lower the best validation loss of medium and large. Saving an earlier checkpoint already avoids their worst final weights, but cannot improve the best result reached. The follow-up therefore targets prediction on unseen pieces, rather than simply reducing training loss. We use validation results to guide this investigation; test data are not used to choose the intervention.

7 Follow-up: can stronger dropout reduce overfitting?

Why regularization, and which kind? The rising validation losses motivated this follow-up; stronger dropout was not part of the initial model-size hypothesis. The baseline already uses dropout 0.1 and AdamW weight decay 0.1, so we investigate whether stronger regularization helps.

Two relevant options are weight decay and dropout. Weight decay shrinks weights during optimization, discouraging large weight values. AdamW separates this shrinkage from the adaptive gradient update [7]. Dropout instead randomly removes activations during training, reducing reliance on particular feature combinations [10]. For drop probability p,

$$\widetilde{h} = \frac{m \odot h}{1-p}, \qquad m_j \sim \text{Bernoulli}(1-p). \tag{3}$$

Here, h is an activation and m is a random zero/one mask. Remaining values are rescaled to keep their expected size unchanged. Dropout is disabled for evaluation. The shared probability affects embeddings, attention probabilities, attention outputs, and MLP outputs.

We chose to increase dropout from 0.1 to 0.2 while keeping weight decay unchanged. This lets us test one setting within the remaining budget. The readings motivate both options, but do not establish which is best for this dataset. We did not measure weight norms or test stronger weight decay, so we cannot claim that insufficient weight decay caused the problem or that dropout is the better remedy. The value 0.2 is a moderate trial, not a known optimum.

Controlled comparison. All three sizes are retrained from random initialization. Within each size, we keep the architecture, seed 1337, AdamW settings, learning-rate schedule, context 1,024, batch 32, validation windows, evaluation frequency, and 7,200-second budget unchanged. Only dropout and output location change. The launcher checks configurations against the saved baselines. Including small checks whether an intervention motivated by medium and large also helps a model with less obvious overfitting. This adds three two-hour runs, reusing the earlier batch-size benchmark.

Success criterion. Compare each size with its own baseline using the lowest recorded validation loss. A smaller training–validation gap alone is insufficient if validation loss worsens. Final loss, time of the best checkpoint, and tokens processed are secondary checks. One shared seed controls initialization, but does not make dropout masks or training paths identical. This is a limited follow-up, not an exhaustive hyperparameter search.

8 Results with stronger dropout

Appendix Figure 2 compares validation loss with dropout 0.1 and 0.2 for each model. The full follow-up curves are also available in the repository's regularization analysis notebook.

Table 6: Effect of dropout on sampled validation losses. Δ best is dropout 0.2 minus 0.1; negative favors the intervention.

	Best validation			Final validation	
Model	$p = 0.1$	$p = 0.2$	Δ best	$p = 0.1$	$p = 0.2$
Small	**1.7652**	1.7665	$+0.0013$	1.7659	1.7665
Medium	1.8018	**1.7872**	-0.0146	1.8892	1.8051
Large	1.8122	**1.7968**	-0.0154	1.9224	1.8305

How the benefit differs by model size

With stronger dropout, medium and large reduce their best validation perplexities by about 1.45% and 1.53%. Medium reaches its best recorded result at 60.1 minutes instead of 40.1, while large reaches it at 60.4 minutes instead of 50.3. Their final validation losses improve by 0.0840 and 0.0919 nats/token. These improvements are larger than the changes in their best losses. Stronger dropout therefore mainly reduces the worsening seen late in training, while also slightly improving the best checkpoints.

The final gaps between training and validation loss fall from 0.5593 to 0.3841 for medium and from 0.6360 to 0.4572 for large. Validation loss still rises after its minimum, so dropout reduces overfitting without removing it. Since we evaluate about every ten minutes, we cannot identify the exact best stopping time.

Small's best loss increases by just 0.0013, or about 0.13% in perplexity. Its final training–validation gap falls from 0.2055 to 0.1444, but validation performance does not improve. This shows why a smaller gap alone does not mean a better model. The recorded early validation curves are worse with stronger dropout for all three sizes, but it appears more useful later for the larger models. The curves support this interpretation, but do not prove which internal features caused the result.

What remains unanswered before testing

Stronger dropout helps medium and large, but neither overtakes small. This suggests that overfitting explains part of their later worsening, while stronger regularization alone does not resolve the size ranking. Small is an unsuccessful intervention: its smaller gap does not produce better validation predictions. Its 0.0013 loss difference is too small to interpret confidently without repeated seeds. We should not describe dropout 0.2 as harmful in general.

The next questions are whether these small improvements repeat across seeds, whether another dropout or weight-decay value helps more, and whether the ranking changes with a different learning rate or equal-token budget. These remain open because those experiments were not run. Testing now addresses a narrower question: how do the validation-selected settings predict unseen pieces under one common scoring method?

How comparable the runs are

The follow-up runs process 4.139B, 1.329B, and 0.507B tokens, close to the baseline counts. A large change in training speed is therefore unlikely to explain the pattern, although equal time does not guarantee equal amounts of training. Small still has the lowest sampled validation loss across both dropout settings. These runs suggest that the benefit of dropout depends on model size; they do not show that 0.2 is always best or better than tuning weight decay. The six training runs use about 12 GPU-hours of loop time, excluding benchmarking, setup, and final evaluation.

9 Final evaluation: what transfers to unseen pieces?

Selection, assumptions, and evaluation history

We select the tested dropout value with the lowest sampled validation loss within each size: **small 0.1, medium 0.2, large 0.2**. Their best checkpoints occur at about 110.1, 60.1, and 60.4 minutes. This compares each size with its selected setting, so the final table does not isolate model size alone. The original equal-dropout baseline remains the direct size comparison.

Validation-based selection assumes these pieces guide performance on similar unseen piano recordings. The separate test split estimates performance after selection [8, Section 5.3]. However, our evaluation was revised: all three dropout-0.2 checkpoints were tested first, then small was replaced with dropout 0.1, which validation had already preferred before test access. Only small inference was repeated; the archive retains the original result. Its old test loss, 1.6970, also ranked first. This checks that the ranking does not depend on the replacement, but cannot restore an untouched pre-test protocol. We report the revision instead of treating the final comparison as fully independent confirmation.

Common scoring and sensitivity to aggregation

Dropout and gradients are disabled. Every token after the first in each piece is scored once, including EOS and partial tails: 3,790,050 validation and 4,334,092 test targets. Consecutive blocks contain at most 1,024 targets and reset context; padding is excluded and pieces never share context. Thus early tokens in a block have shorter histories. This is reproducible full coverage, but not full sliding-context likelihood. We rescore validation identically to test rather than comparing test directly with the sampled training-time estimator.

Table 7: Validation-selected checkpoints under the same full-coverage scoring rule. Loss is cross-entropy in nats/token.

Model	Dropout	Full validation loss	Test loss	Test perplexity
Small	0.1	**1.7581**	**1.6935**	**5.4385**
Medium	0.2	1.7848	1.7179	5.5730
Large	0.2	1.7926	1.7277	5.6279

The test loss differences relative to small are 0.0244 and 0.0342 nats/token, or about 2.5% and 3.5% higher perplexity. These are modest prediction gaps, despite roughly fivefold and seventeenfold parameter increases. Small also wins after full-coverage validation rescoring, so switching from sampled windows does not reverse the ranking. Test loss is lower for every model than full validation loss. Different pieces may differ in difficulty; this is not evidence that testing improves weights or a reason to tune on test data.

Overall scores weight tokens equally, giving longer pieces more influence. We therefore checked the same saved predictions with each piece weighted equally (Appendix Table 8). The losses are 1.7090, 1.7335, and 1.7432: the ordering is unchanged. More strongly, small has lower loss than both alternatives on each of the 177 matched test pieces. This rules out a ranking caused only by a few long pieces in this evaluation. It does not establish robustness across random seeds or independent datasets: all pieces share the same fitted checkpoints, and model-training variation has not been measured. These checks are exploratory descriptions of inspected test outputs, not additional model-selection evidence.

Which events explain the gap?

Appendix Table 9 separates the target types. To interpret their importance, each category's mean-loss difference is multiplied by its share of targets. Note starts account for about 0.0172 of medium's 0.0244 total gap and 0.0231 of large's 0.0342 gap: approximately 70% and 68%. This localizes most of the disadvantage to predicting note starts, rather than a uniform weakness across events. It motivates investigating pitch-pattern learning, but is not proof of the internal cause.

Time shifts make up 46.13% of targets. Both larger models slightly improve on them, and large also improves on pedal events, but these gains do not offset its note-related losses. A single aggregate score can therefore hide useful differences. Conversely, EOS is only 177 targets (0.0041%) and contributes roughly 0.00084 nats/token to small's total loss despite its mean loss of 20.51. Removing EOS would barely change the average, while generated pieces might still fail to end appropriately. Prediction loss should therefore be paired with termination and listening checks before claiming musical quality.

The fixed training-window rule may contribute: EOS appears only as the last target of the final valid window, whereas interior tokens can appear in many windows. This is an untested sampling explanation, not established evidence that the architecture cannot learn endings. A deliberate tail-sampling comparison would need explicit weighting to avoid silently changing the objective. Code and saved evidence support reproduction, but checkpoints are excluded from Git; independent inference requires retrieving the original checkpoints or retraining.

10 Conclusions and the next experiment

What the evidence establishes

The initial hypothesis that larger models would predict better is not supported within two A10 hours per run. Small wins the equal-dropout baseline and the final comparison of validation-selected settings. This is a useful negative result about resource allocation: more capacity is not sufficient to improve this pipeline. It is not a general claim that small models outperform large models. Size, updates, repeated exposure, and the suitability of the shared learning rate all change together.

The follow-up gives a more specific result. Raising dropout improves medium/large best validation losses by 0.0146/0.0154, while reducing their final losses by 0.0840/0.0919. Its main effect is therefore to reduce late worsening, with smaller gains at the checkpoints actually selected. Dropout does not close the baseline disadvantage. For small, reducing the

train–validation gap produces no improvement in best validation loss. The intervention's value depends on the measured prediction objective, not on making curves look closer. One seed cannot tell us whether small's 0.0013 change is reproducible.

Why more training is not automatically the answer

The baseline processes roughly 125.4/40.1/15.2 training-corpus equivalents for small/medium/large. These are exposure ratios under replacement sampling, not complete ordered epochs or new musical information. Muennighoff et al. find diminishing returns from repeated data in data-constrained language models [11]. This makes diversity a plausible limitation to investigate here. Their text-model repetition thresholds do not transfer directly to our event vocabulary, overlapping windows, or small architectures. Our curves do not prove a data-scaling law, and the larger models' lower training losses do not by themselves distinguish memorization from optimization effects.

Likewise, the shared time-based schedule equalizes the learning rate at the same elapsed time, not after the same number of updates. Validation and saving advance that clock, and ten-minute checks may miss each model's true best state. Reused validation windows can also favor their particular contents. Additional seeds, broader validation coverage, and comparable per-size tuning budgets are needed before treating the ranking as stable. The 177-piece consistency check addresses evaluation composition, not these training uncertainties.

Which future iteration is best justified?

First, improve data use on the small baseline. Test training-only pitch transposition while leaving validation/test pieces unchanged. Shift note starts and matching endings together; reject shifts outside the playable range rather than clipping pitches. This preserves intervals while varying absolute pitch, but assumes transposed performances remain useful examples and may alter key/register statistics. Huang et al. use pitch/time augmentation in their piano experiments [12, Section 4.2]; this supports trying it, not a predicted gain for our tokenizer. Test augmentation separately from architecture changes so any improvement remains interpretable.

For an architecture change, prefer a small relative-attention model over MoE or retrieval. Hoffmann et al.'s improved-architecture discussion covers conditional computation and retrieval, rather than establishing a winner for limited music data [9, Section 2]. MoE reduces the active fraction of weights but adds routing and expert-management costs; Switch Transformers documents complexity and training-instability concerns [13]. Our larger dense models already fit training well, so adding expert capacity does not directly address the observed failure to generalize. Retrieval would also require a useful music index and a training-only retrieval boundary; indexing the same small corpus does not create new performances. Neither option is ruled out, but both add less-targeted complexity at this stage.

Relative attention instead lets the small model use distances between events rather than relying only on absolute token positions. Music Transformer reports benefits on expressive piano event sequences [12], a closer task match than the large text-model evidence for MoE/retrieval. This is our preferred architectural hypothesis, not a demonstrated remedy for overfitting. Event distance is not elapsed musical time, and altered attention could lose optimized-kernel speed or use more memory. Start with the same 1,024-token context; do not simultaneously increase context or change dropout.

Make the next comparison falsifiable. First repeat the small baseline, then compare augmentation alone and relative attention alone, each with the same two-hour A10 budget, several seeds, identical validation windows, and comparable tuning effort. Report best validation loss, throughput, memory, parameter count, and actual tokens processed. A loss gain that disappears under equal time because attention is slower would reject its practical benefit for this project. Follow with a separately scheduled equal-token comparison to study learning efficiency. Check note/pitch families, tails/EOS, and matched-prompt MIDI continuations as secondary diagnostics. Because test outputs have been inspected, further choices must use validation; fresh confirmation needs a new holdout. These are proposed experiments, not completed results.

References

[1] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, and Douglas Eck. Enabling factorized piano music modeling and generation with the MAESTRO dataset. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1810.12247.

[2] Magenta. The MAESTRO dataset, version 3.0.0, 2026. URL https://magenta.tensorflow.org/datasets/maestro. Dataset documentation, accessed 8 November 2026.

[3] HuggingFaceTB. The smol training playbook: The secrets to building world-class LLMs, 2025. URL https://huggingface.co/spaces/HuggingFaceTB/smol-training-playbook. Practitioner guide; accessed 12 November 2026.

[4] NVIDIA. NVIDIA A10 tensor core GPU, 2026. URL https://www.nvidia.com/en-us/data-center/products/a10-gpu/. Product specifications, accessed 8 November 2026.

[5] Andrej Karpathy. nanoGPT, 2026. URL https://github.com/karpathy/nanoGPT. Source implementation, accessed 8 November 2026.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. URL https://arxiv.org/abs/1706.03762.

[7] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

[8] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. URL https://www.deeplearningbook.org/contents/ml.html. Chapter 5.

[9] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2203.15556.

[10] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. URL https://jmlr.org/papers/v15/srivastava14a.html.

[11] Niklas Muennighoff, Alexander M. Rush, Boaz Barak, Teven Le Scao, Aleksandra Piktus, Nouamane Tazi, Sampo Pyysalo, Thomas Wolf, and Colin Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/9d89448b63ce1e2e8dc7af72c984c196-Abstract-Conference.html.

[12] Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck. Music transformer: Generating music with long-term structure. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1809.04281.

[13] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.

A Learning curves and detailed test results

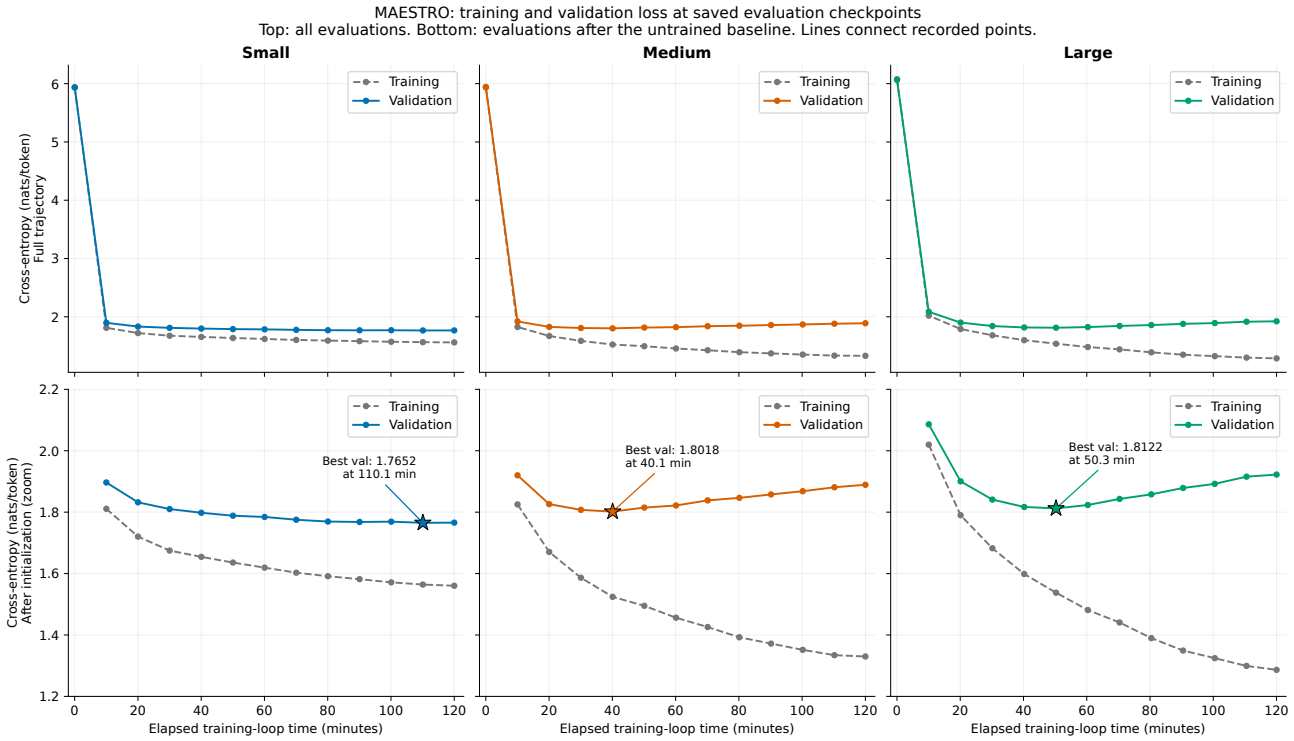


Figure 1: Baseline learning curves. Dashed grey: sampled training loss; solid colors: sampled validation loss. Stars mark best recorded validation checkpoints. The bottom row omits initialization. Lines connect recorded checks; they do not add observations.

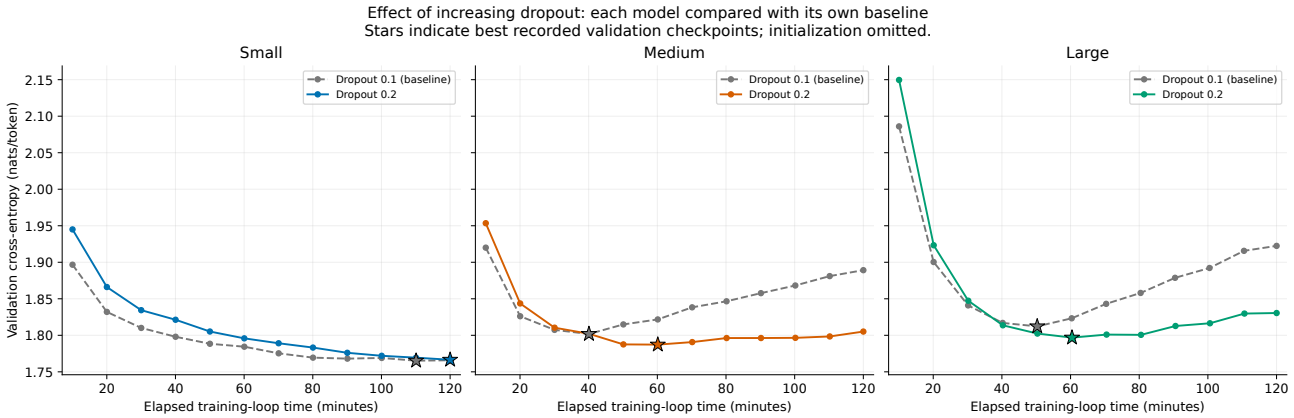


Figure 2: Follow-up prompted by the baseline curves: dropout 0.1 (grey dashed) versus 0.2 (colored solid). Each size is compared with its own baseline; stars mark its best recorded validation losses. Medium and large improve, while small does not.

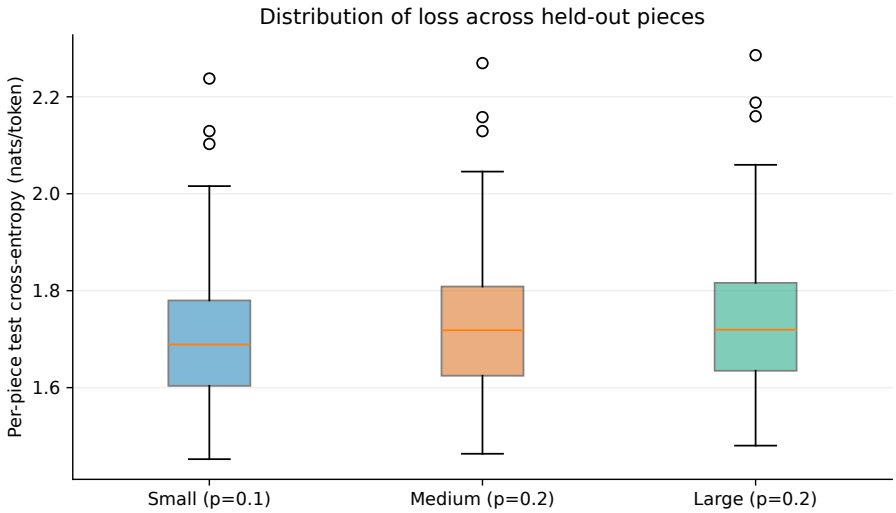


Figure 3: Test-loss distributions for the selected settings: small dropout 0.1, medium/large dropout 0.2. Each model is scored on the same 177 pieces. These distributions give each piece equal weight, while the overall loss gives each token equal weight. They do not measure variation across training seeds.

Table 8: Exploratory checks on the same saved test predictions. Equal-piece loss gives every piece the same weight. The last column counts pieces where small has lower loss. These are not repeated training runs.

Model	Token-weighted loss	Equal-piece loss	Small wins / 177
Small	1.6935	1.7090	–
Medium	1.7179	1.7335	177
Large	1.7277	1.7432	177

Table 9: Per-event-family test cross-entropy for the selected configurations. Counts are shared across models.

Family	Targets	Small ($p = 0.1$)	Medium ($p = 0.2$)	Large ($p = 0.2$)
NOTE_ON	741,410	1.7608	1.8615	1.8960
NOTE_OFF	741,410	1.1307	1.1544	1.1721
TIME_SHIFT	1,999,264	1.6817	1.6804	1.6805
VELOCITY	741,410	2.2357	2.2538	2.2692
PEDAL	110,421	1.5642	1.5862	1.5194
EOS	177	20.5116	21.6459	20.2792

The event counts are identical across models. Each category's loss still uses probabilities over the full vocabulary. EOS has high loss but only 177 targets, so poor prediction of endings has little effect on overall loss. These diagnostics do not measure audio or listening quality. Additional figures and analysis notebooks remain available in the repository.